

Rapport d'Audit Technique

Smart Shepherd - Plateforme Admin IoT/IA

Audit architecture, produit et ingenierie

29 mai 2026

Table des matières

1	Resume executif	3
2	Methodologie et perimetre	3
2.1	Sources inspectees	3
2.2	Limites	3
3	1. Identification des technologies	3
3.1	Frontend	3
3.2	Backend	4
3.3	IA et data science	4
3.4	Infra et exploitation	4
3.5	LoRa / LoRaWAN	4
4	2. Analyse complete des pages	4
4.1	Pages identifiees	4
4.2	Evaluation fonctionnelle par domaine	5
4.3	UX/UI	5
5	3. Analyse speciale IA	5
5.1	Constat de realite IA	5
5.2	Modes detectes	5
5.3	Points critiques IA	6
5.4	Recommandations IA prioritaires	6
6	4. Analyse de la simulation	6
6.1	Etat actuel	6
6.2	Limites	6
6.3	Cible recommandee	6
7	5. Analyse du dashboard administratif	6
7.1	Architecture frontend	6
7.2	Roles et permissions	7
7.3	Securite	7

8	6. Detection des problemes	7
8.1	Problemes critiques	7
8.2	Problemes majeurs	7
8.3	Problemes moyens	7
9	7. Fonctionnalites manquantes pour un niveau SaaS IoT pro	7
10	8. Analyse architecture logicielle	8
10.1	Evaluation	8
10.2	Architecture cible recommandee	8
10.3	Architecture cible enterprise	8
10.4	Contrats, schemas et standardisation	8
10.5	Architecture temps reel cible	8
10.6	IA, MLOps et gouvernance	9
10.7	Simulation enterprise	9
10.8	Observabilite et DevOps	9
10.9	Device management	9
10.10	Migration strategy	9
10.11	Backlog critique priorise	10
11	9. Evaluation professionnelle (notes /10)	10
12	10. Roadmap d'amelioration recommandee	10
12.1	Phase 1 (0-30 jours)	10
12.2	Phase 2 (30-90 jours)	11
12.3	Phase 3 (90-180 jours)	11
13	Conclusion	11

1. Resume executif

Ce rapport presente une analyse technique complete du dashboard administratif et de la plateforme Smart Shepherd sur la base du code source disponible.

Conclusion courte : la plateforme est avantee et convaincante en demonstration (temps reel, carte, analytics, IA, simulation), mais reste hybride entre MVP avance et produit SaaS industriel.

Forces majeures :

- Stack moderne coherente pour un produit IoT (React + Node + MongoDB + MQTT + Socket.IO + FastAPI).
- Dashboard riche avec nombreuses pages metier et UX globalement solide.
- Simulation IoT detaillee et parametres avances de scenario.
- Presence d'une base DevOps (Docker Compose, Prometheus, Grafana, Alertmanager).

Risques majeurs :

- Securite et authentication heterogenes (plusieurs middlewares auth, bypass dev, roles non uniformes).
- IA multi-chemin (Node local, FastAPI, LLM externe) avec fallback mock partiellement opaque.
- Contrats API et evenements temps reel non totalement uniformises.
- Presence de comportements fallback/mocks pouvant masquer des incidents en production.

2. Methodologie et perimetre

2.1. Sources inspectees

L'analyse couvre principalement :

- Frontend : routage, pages, hooks et services temps reel.
- Backend : API Express, routes metier, MQTT, Socket.IO, auth.
- Microservice IA Python FastAPI.
- Infrastructure Docker (dev/prod) et observabilite.

2.2. Limites

- Pas de capture d'ecran exploitee dans ce passage.
- Pas de benchmark de charge execute dans ce rapport.
- Evaluation basee sur le code et la structure projet.

3. 1. Identification des technologies

3.1. Frontend

- React 18 + TypeScript + Vite.
- React Router v6 pour les routes applicatives.
- Zustand pour l'etat global IoT et alertes.

- React Query pour requetes API et mutations.
- Tailwind CSS + composants UI custom.
- Chart.js/Recharts pour visualisation.
- Leaflet/React-Leaflet + Geoman pour cartes/geofence.
- MQTT client navigateur et Socket.IO client.

3.2. Backend

- Node.js + Express (ESM).
- MongoDB via Mongoose.
- Redis (cache conditionnel).
- Socket.IO serveur pour push realtime.
- MQTT client serveur pour ingestion/emission flux IoT.

3.3. IA et data science

- Service IA Node (tfjs-node conditionnel, logique prediction et scoring).
- Microservice FastAPI (TensorFlow/XGBoost, endpoints IA).
- LLM externe Anthropic avec fallback local heuristique.

3.4. Infra et exploitation

- Docker Compose dev/prod.
- Mosquitto, Mongo, Redis.
- Prometheus, Alertmanager, Grafana.
- Nginx reverse proxy.

3.5. LoRa / LoRaWAN

- Presence d'un dossier firmware ESP32 avec bibliotheques LoRaWAN.
- Pas d'integration backend explicite complete type network server LoRaWAN -> decoder -> pipeline data.

Verdict coherence techno : choix global logique et pertinent pour une plateforme IoT intelligente; les points faibles proviennent surtout de l'uniformisation d'architecture et de gouvernance runtime.

4. 2. Analyse complete des pages

4.1. Pages identifiees

- Dashboard, Map, Animals, Animal Profile, Compare, Agenda, Alerts.
- Anomalies, Analytics, AI Dashboard.
- Users, Hardware, Settings.
- Admin : AI Settings, Labelling.

4.2. Evaluation fonctionnelle par domaine

Operations :

- Dashboard : KPIs, widgets realtime, alertes, mini-map, meteo.
- Map : supervision geospatiale, geofencing, edition de zones.
- Animals : liste virtualisee, tri/filtre, score sante, navigation detail.
- Alerts : centre de triage realtime.
- Agenda : planification vet.

Intelligence :

- Analytics : heatmap, batterie, tendances historiques, export.
- Anomalies : registre + analyse IA asynchrone.
- AI Dashboard : vue predictive, severite, recommandations.

Administration :

- Users : CRUD avec optimistic update (et mode mock selon config).
- Hardware : flotte colliers, etat derive du store.
- Settings/AI Settings : configuration simulation, seuils, canaux notifications.
- Labelling : annotation terrain alimentee par les alertes et le troupeau live, avec fallback store en secours.

4.3. UX/UI

- Niveau visuel global bon a tres bon (hierarchie, composants, skeletons, badges severite).
- Navigation claire via piliers metier.
- Risque de surcharge informationnelle sur certaines vues denses.
- Certains fallbacks UX peuvent masquer des incidents techniques reels.

5. 3. Analyse speciale IA

5.1. Constat de realite IA

La plateforme propose une vraie couche IA, mais elle fonctionne selon plusieurs modes selon la disponibilite des dependances, des modeles et des cle API.

5.2. Modes detectes

- IA locale Node (prediction/scoring).
- IA microservice FastAPI (health-check, battery-rul, position).
- IA generative externe (Anthropic) pour synthese.
- Fallback heuristique locale en cas d'echec API/cle absente, expose via un mode IA explicite.

5.3. Points critiques IA

- Risque de confusion produit fortement réduit : le backend expose maintenant un `model_mode` explicite (anthropic/local/local-cache).
- Endpoints avec retour success/fallback toujours a surveiller, mais la transparence d'incident est meilleure qu'avant.
- Absence d'une gouvernance MLOps complete (versioning modele, quality gates, drift monitoring).

5.4. Recommandations IA prioritaires

1. Standardiser la reponse IA avec un champ obligatoire `model_mode` (real/mock/fallback).
2. Centraliser l'inference via FastAPI ; garder Node en orchestrateur et policy layer.
3. Mettre en place registre de modeles + metriques qualite + seuils d'acceptation.
4. Isoler IA explicative (LLM) de l'IA decisionnelle (scoring risque).

6. 4. Analyse de la simulation

6.1. Etat actuel

- Simulation riche (nombre animaux, vitesse, densite, scenarios, batterie, alertes).
- Simulation encore principalement centree frontend/store, avec un pipeline temps reel partiellement unifie.
- Bonne valeur pour demo, UX tests, essais fonctionnels rapides.

6.2. Limites

- Pas de digital twin backend canonique.
- Realisme biologique/comportemental limite par des regles heuristiques.
- Rejouabilite et tracabilite simulation encore perfectibles.

6.3. Cible recommandee

1. Moteur de simulation backend event-driven.
2. Publication simulation via MQTT sur le meme pipeline que le reel.
3. Scenarios parametrables avec `run_id` et seeds reproductibles.
4. Correlation simulation avec meteo, geofence, pannes capteurs et anomalies IA.

7. 5. Analyse du dashboard administratif

7.1. Architecture frontend

- Organisation modulaire globalement bonne (pages, composants, hooks).
- Etat global coherent via Zustand, mais certaines logiques restent dupliquees (MQTT/realtime/service).
- Presence d'une forte couche de presentation avancee.

7.2. Roles et permissions

- Intention RBAC visible cote UI.
- Incoherences role naming detectees (admin/ADMIN/super_admin selon couches).
- La securite ne doit pas dependre de la seule couche UI.

7.3. Securite

- Corrections recentes sur JWT_SECRET positives.
- Mais coexistence de plusieurs middlewares auth et bypass dev a controler strictement.

8. 6. Detection des problemes

8.1. Problemes critiques

1. Heterogeneite des middlewares d'authentification et des conventions roles.
2. Bypass dev potentiellement actif selon environnement et routes.
3. Evenements realtime non completement alignes entre emit backend et ecoute frontend.

8.2. Problemes majeurs

1. Contrats API non uniformes (formes de payload variees).
2. Multiples fallback/mocks en chemins utilisateur sans indicateur unifie.
3. Dette de standardisation du pipeline IA.

8.3. Problemes moyens

1. Couplage fort de certains composants pages/services.
2. Presence de logiques historiques/optionnelles qui complexifient la maintenance.
3. Documentation fonctionnelle riche mais parfois en decalage avec execution runtime.

9. 7. Fonctionnalites manquantes pour un niveau SaaS IoT pro

- RBAC centralise avec policy engine unique.
- Audit logs complets (auth, actions admin, config, IA).
- Device management industriel : provisioning, rotation credentials, OTA, inventaire firmware reel.
- Observabilite avancee : traces distribuees, SLO, runbooks.
- MLOps : monitoring de drift, performance modele, dataset lineage.
- Gouvernance multi-tenant stricte et retention politique.
- Exports reglementaires et mode degrade explicite.

10. 8. Analyse architecture logicielle

10.1. Evaluation

- Modularite frontend : bonne.
- Backend : monolithe enrichi avec extensions ; viable mais a cadrer.
- Event-driven : present mais non strictement unifie.
- IA microservice : bonne direction, integration encore hybride.

10.2. Architecture cible recommandee

1. API Gateway + Auth service unique + RBAC policy centralisee.
2. Ingestion IoT unique (MQTT) + normalisation schema + event bus interne.
3. Service IA central d'inference versionnee.
4. Contrats API/evenements versionnees (OpenAPI + event schema).
5. Frontend branche sur etats data explicites (live/simulated/cached/mock).

10.3. Architecture cible entreprise

- Couche d'entree : API Gateway, Auth Service, rate limiting et politiques de securite.
- Couche metier : IoT Ingestion Service, Alert Service, Analytics Service, Device Service.
- Couche IA : FastAPI centralise, model registry, inference service, monitoring des modeles.
- Couche temps reel : MQTT Broker, normalizer, event bus, WebSocket Gateway, subscriptions front.
- Couche donnees : MongoDB pour les objets metier, PostgreSQL pour les registres structurants, Redis pour cache/queues.
- Couche front : dashboard modulaire, design system, state management explicite live/sim/mock/fall

10.4. Contrats, schemas et standardisation

- Toutes les APIs doivent exposer `{success, data, error, metadata}`.
- Les erreurs doivent etre normalisees par code, message, details et correlation_id.
- Les evenements MQTT et WebSocket doivent partager le meme schema canonique.
- Les DTO doivent etre versionnees (v1, v2) avec validation Zod/Joi cote backend.
- OpenAPI doit servir de source de verite pour les routes HTTP.

10.5. Architecture temps reel cible

1. IoT Device publie vers un topic MQTT normalise.
2. Broker MQTT alimente un event normalizer avec idempotence et correlation id.
3. Event bus distribue les evenements vers alerting, analytics, IA et websocket.
4. WebSocket Gateway pousse vers le dashboard uniquement les vues abonnees.
5. Le frontend affiche clairement les modes LIVE, SIMULATION, MOCK et FALLBACK.

10.6. IA, MLOps et gouvernance

- Un service IA decisionnel doit produire health score, anomaly score et behavior analysis.
- Un service IA explicatif doit generer les syntheses textuelles et recommandations.
- Le model registry doit conserver version, dataset, metrics, date de promotion et roll-back target.
- Les predictions doivent remonter un `confidence score` et un `model_mode`.
- Les modeles doivent etre surveilles sur drift, calibration, latency et taux d'erreur.

10.7. Simulation enterprise

- La simulation doit devenir un moteur backend event-driven avec `run_id` et seeds reproductibles.
- Chaque scenario doit simuler GPS, temperature, rythme cardiaque, batterie, geofence et panne reseau.
- Les evenements de simulation doivent passer par le meme pipeline MQTT que les donnees reales.
- Le dashboard doit montrer explicitement la provenance des donnees : LIVE, SIMULATION, MOCK ou FALLBACK.

10.8. Observabilite et DevOps

- Metriques : latence API, latence MQTT, files d'attente, erreurs IA, taux d'alertes, uptime.
- Logs : structure JSON, `correlation_id`, audit logs admin, logs d'inference IA.
- Traces : OpenTelemetry pour router les requetes entre gateway, services et broker.
- Alerting : Prometheus + Grafana + Alertmanager + notifications critiques.
- Protection : backups, rollback, health checks, readiness/liveness et rotation secrets.

10.9. Device management

- Inventaire firmware et hardware par device.
- Provisioning, rotation de certificats, stockage des identites de devices.
- Heartbeat, batterie, connectivite, geofence, statut de securite.
- OTA updates avec suivi d'etat et rollback firmware.

10.10. Migration strategy

1. Phase A : introduire Auth Service, schema API commun et modele de reponse standard.
2. Phase B : unifier les evenements realtime et isoler le pipeline IA.
3. Phase C : deplacer la simulation vers le backend et brancher le dashboard sur les sources canoniques.
4. Phase D : industrialiser observabilite, MLOps et device management.

10.11. Backlog critique priorise

Chantier	Priorite	Effet attendu
Auth/RBAC centralise	P0	Une seule source d'autorite pour les roles, permissions et tokens
API/event contracts	P0	Reduction des incoherences de payload et des integrations fragiles
Realtime unifie	P0	Moins de duplications et de disconnects inter-pages
IA centralisee	P1	Modele explicite, traçable et monitorable
Simulation backend	P1	Donnees reproductibles et pipeline canonique
Observabilite	P1	Diagnostic rapide des incidents et mesure SLA/SLO
Device management	P2	Ready pour exploitation terrain serieuse
CI/CD et security hardening	P2	Livraison plus sure et plus frequentable

11. 9. Evaluation professionnelle (notes /10)

Axe	Note	Commentaire court
Architecture logicielle	7.2	Bonne base, besoin d'unification
Frontend admin	8.4	Niveau visuel et UX eleve
Backend API	6.8	Fonctionnel mais heterogene
IA (maturite produit)	6.1	Potentiel fort, gouvernance insuffisante
UX/UI	8.2	Solide et moderne
Temps reel IoT	7.0	Bon socle, schemas a aligner
Securite applicative	5.9	Chantiers prioritaires restants
Qualite produit SaaS	7.0	MVP avance, pas encore enterprise
Credibilite startup tech demo	8.0	Convaincant pour demo/invest
Niveau ingenierie global	7.4	Bon niveau, dette d'industrialisation

12. 10. Roadmap d'amelioration recommandee

12.1. Phase 1 (0-30 jours)

1. Unifier auth/RBAC et conventions roles.
2. Standardiser schema evenements realtime.

3. Rendre explicite le mode de donnees affiche (live/sim/fallback).
4. Supprimer fallback silencieux critiques en production.

12.2. Phase 2 (30-90 jours)

1. Versionner les contrats API et evenements.
2. Deplacer simulation vers backend event-driven.
3. Unifier pipeline IA et instrumenter metriques qualite.
4. Ajouter audit logs structurants et traçabilite actions.

12.3. Phase 3 (90-180 jours)

1. Multi-tenant robuste et securite zero-trust inter-services.
2. Device twin + OTA workflow complet.
3. Observabilite complete (metrics/logs/traces/SLO).
4. CI/CD qualite : tests integration, tests contrat, scans securite.

13. Conclusion

Smart Shepherd dispose d'une base technique ambitieuse et credible. Le produit est deja fort en UX et demonstration metier. Pour atteindre un standard SaaS IoT/IA enterprise, la priorite n'est pas d'ajouter des ecrans mais de renforcer la coherence transversale : securite, gouvernance des contrats, fiabilite realtime et industrialisation IA.

Decision architecturale conseillée : conserver la stack actuelle, mais lancer un programme d'unification technique sur 3 phases avec jalons de qualite mesurables.